

Aspect-Aware Sentiment Analysis of Malaysian Telecommunications and Broadband Feedback

USER MANUAL

TNL6323 — Natural Language Processing

Group Project (40%)

Software version: 1.0

Document date: July 2026

This manual explains how to install, run, and use the source code of the Malaysian telco and broadband sentiment-analysis system.

Table of Contents

1	Introduction.....	3
1.1	What the software does.....	3
1.2	Intended audience and how to read this manual.....	3
2	System Requirements.....	3
3	Installation and Setup.....	4
3.1	Step 1 — Obtain the source code.....	4
3.2	Step 2 — Create and activate a virtual environment.....	4
3.3	Step 3 — Install the dependencies.....	4
3.4	Step 4 — Optional components.....	4
3.5	A note on regenerated files.....	5
4	Repository Contents.....	5
4.1	Pipeline scripts.....	5
4.2	Data-collection scripts.....	5
5	Quick Start.....	6
6	Running the Full Pipeline.....	6
6.1	Fine-tuning options.....	6
6.2	The flagship degradation experiment.....	7
7	Using the Application.....	7
7.1	Launching the application locally.....	7
7.2	Analysing a review.....	7
7.3	The deployed web application.....	8
8	Input and Output Reference.....	8
8.1	Data format.....	8
8.2	Output files.....	8
9	Troubleshooting.....	9
10	Glossary.....	9
11	Further Documentation.....	10

1 Introduction

This document is the user manual for the sentiment-analysis system developed for the TNL6323 Natural Language Processing group project. The system analyses customer feedback about Malaysian telecommunications and broadband services, and is intended for graders, examiners, and any developer who wishes to install the source code, reproduce the reported results, or operate the accompanying web application.

1.1 What the software does

Given a customer review or a forum post — written in English, Bahasa Melayu, or Manglish (code-switched Malay-English) — the system performs two tasks. First, it classifies the overall sentiment of the text into one of three classes: negative, neutral, or positive. Second, it identifies which of five fixed service aspects the text discusses — coverage, speed, billing, customer support, and application usability — and reports a sentiment for each aspect that is mentioned.

Two classification models are provided and compared. The baseline model uses classical TF-IDF features with Logistic Regression and a Linear Support Vector Machine. The advanced model is a fine-tuned XLM-RoBERTa transformer, which handles multilingual and code-switched text directly. Every model is evaluated across three tiers of increasingly out-of-domain data — Play Store reviews (in-domain), Trustpilot reviews (cross-platform), and Reddit or Lowyat forum posts (cross-domain). The change in accuracy across these tiers, and whether that change is uniform across the five aspects, is the principal analytical result of the project.

1.2 Intended audience and how to read this manual

The manual is task-oriented and assumes only basic familiarity with a command line and with Python. Readers who simply want to see the system produce a result should follow Section 3 (installation) and then Section 5 (quick start). Readers who wish to reproduce the full set of experimental results should additionally follow Section 6. Readers who only want to operate the interactive web application should read Section 7. A glossary of domain terms is provided in Section 10.

2 System Requirements

The software is written in Python and runs on Windows, macOS, and Linux. The requirements below describe what is needed to install the dependencies, run the pipeline, and launch the application.

Component	Requirement
Operating system	Windows 10/11, macOS, or Linux
Python	Version 3.10 or later (the deployed web app is pinned to Python 3.12)
Disk space	Approximately 2 GB, of which about 1.1 GB is the fine-tuned XLM-RoBERTa model
Memory	8 GB RAM recommended for running the transformer model on CPU
Processor (GPU)	A GPU is required only for fine-tuning the transformer. Google Colab's free GPU tier is sufficient. The baseline

	model, the evaluation, and the web application all run on CPU.
Internet	Required only to install dependencies, download the base model, or run the data scrapers

All Python dependencies are declared in the file `requirements.txt` and are installed in a single step (Section 3). The main libraries are pandas and NumPy for data handling, scikit-learn for the baseline model, PyTorch together with Hugging Face Transformers and Datasets for the transformer, and Gradio for the web application.

3 Installation and Setup

The following steps install the software into an isolated Python virtual environment. All commands are entered at a terminal (Command Prompt or PowerShell on Windows; a shell on macOS or Linux).

3.1 Step 1 — Obtain the source code

Clone the repository and change into the project directory:

```
git clone <repository-url>
cd tn16323-project
```

3.2 Step 2 — Create and activate a virtual environment

Create a virtual environment named `.venv`:

```
python -m venv .venv
```

Activate it. On Windows:

```
.venv\Scripts\activate
```

On macOS or Linux:

```
source .venv/bin/activate
```

3.3 Step 3 — Install the dependencies

With the environment active, install all required packages:

```
pip install -r requirements.txt
```

3.4 Step 4 — Optional components

The following steps are needed only for specific tasks and may be skipped by most users. The Trustpilot scraper drives a headless browser, which must be installed once:

```
python -m playwright install chromium
```

The Reddit scraper requires API credentials. Copy the provided template to a file named `.env` and fill in your credentials obtained from <https://www.reddit.com/prefs/apps>:

```
copy .env.example .env      (Windows)
cp .env.example .env        (macOS / Linux)
```

3.5 A note on regenerated files

A fresh clone of the repository includes the hand-labelled datasets under `data/labeled/`, but it does not include the trained models or the raw and cleaned data, which are excluded from version control because of their size. The labelled data is sufficient to train the models and reproduce the results; the trained models are regenerated locally by running the pipeline in Section 6. The quickest way to obtain a working model is described next in Section 5.

4 Repository Contents

The source code is organised into scrapers that collect data, a processing and modelling pipeline under `src/`, and the web application under `app/`. The tables below summarise the scripts a user is most likely to run and what each one reads and writes. Every script is a self-contained program invoked with `python <path>`, and must be run from the repository root so that its relative file paths resolve correctly.

4.1 Pipeline scripts

Script	Purpose	Reads → Writes
<code>src/preprocess.py</code>	Merge, clean, deduplicate, and language-detect the raw data; drop reviews shorter than five words	<code>data/raw/</code> → <code>data/cleaned/all_sources.csv</code>
<code>src/baseline.py</code>	Train the TF-IDF baseline (Logistic Regression and Linear SVM)	labelled data → <code>models/baseline_lr.pkl</code> , <code>baseline_svm.pkl</code>
<code>src/finetune_xlmr.py</code>	Fine-tune the XLM-RoBERTa transformer (requires a GPU)	<code>labeled_main_train.csv</code> → <code>models/xlmr_final/</code>
<code>src/aspect_sa.py</code>	Rule-based extractor for the five service aspects	labelled data → <code>labeled_main_with_aspects.csv</code>
<code>src/evaluate.py</code>	Produce the three-tier accuracy comparison table	eval sets + models → <code>models/eval_results.csv</code>
<code>src/aspect_degradation.py</code>	Flagship experiment: accuracy per model, tier, and aspect	eval sets + models → <code>models/aspect_degradation.csv</code> + heatmaps

4.2 Data-collection scripts

The scrapers reproduce the data-collection stage. They are optional for reproducing results, because the labelled evaluation data is already provided in the repository.

Script	Source	Note
--------	--------	------

scrapers/play_store.py	Google Play Store reviews	Primary training source
scrapers/reddit.py	Reddit posts	Requires API credentials in .env
scrapers/trustpilot.py	Trustpilot reviews	Requires the Chromium browser (Section 3.4)
scrapers/lowyat.py	Lowyat forum posts	Optional

5 Quick Start

This section gives the shortest path from a fresh installation to a working result. It trains the lightweight baseline model (which runs in a minute or two on CPU), evaluates it, and then launches the web application. All commands are run from the repository root with the virtual environment active.

Train the baseline model:

```
python src/baseline.py
```

This writes models/baseline_lr.pkl and models/baseline_svm.pkl. Evaluate the model across the three tiers:

```
python src/evaluate.py
```

This prints the accuracy comparison and writes models/eval_results.csv. Note that evaluate.py requires a trained baseline model to be present; if models/baseline_lr.pkl does not yet exist, run the training step above first.

Finally, launch the web application (see Section 7 for details):

```
python app/app.py
```

6 Running the Full Pipeline

To reproduce the complete set of results, run the pipeline in the order below from the repository root. Each stage writes its outputs into data/ or models/, which the following stage consumes.

```
python src/preprocess.py      # clean and deduplicate the raw data
python src/baseline.py       # train the TF-IDF baseline
python src/finetune_xlrmr.py  # fine-tune XLM-RoBERTa (needs a GPU)
python src/aspect_sa.py      # tag rows with the five service aspects
python src/evaluate.py       # three-tier accuracy table
python src/aspect_degradation.py # per-aspect degradation + heatmaps
```

6.1 Fine-tuning options

The transformer fine-tuning script accepts a small number of command-line options. Running it with no options trains for five epochs on the 960-row training split:

Option	Effect
--train <path>	Path to the training-split CSV (default:

	data/labeled/labeled_main_train.csv)
--epochs <n>	Number of training epochs (default: 5)
--smoke	A one-step sanity run on a handful of rows; runs on CPU and is used to verify the environment without training a real model

Fine-tuning requires a GPU. If no local GPU is available, use the supplied Colab notebook `notebooks/finetune_xlmr_colab.ipynb`: upload the training split, install the transformer libraries, run the notebook, and download the resulting `models/xlmr_final/` folder back into the local repository before running the evaluation.

6.2 The flagship degradation experiment

The script `src/aspect_degradation.py` measures, for every combination of model, evaluation tier, and aspect, the accuracy on the subset of rows that mention that aspect. It writes `models/aspect_degradation.csv` and, when the matplotlib library is available, a heatmap image per model. This script is designed to run safely even when some models or evaluation sets are missing: it always writes a valid output file and reports which artefacts were absent rather than stopping with an error.

7 Using the Application

The system includes an interactive web application built with Gradio. It lets a user type or paste a review and immediately see the predicted sentiment and the per-aspect analysis.

7.1 Launching the application locally

From the repository root, with the virtual environment active, run:

```
python app/app.py
```

The application starts a local web server and prints a URL, normally `http://127.0.0.1:7860`. Open that address in a web browser. By default the application loads the fine-tuned model from `models/xlmr_final`; if that folder is not present, the interface still opens but reports that the model is not loaded. To point the application at a model published on the Hugging Face Hub instead, set the `HF_MODEL_ID` environment variable to the model repository identifier before launching.

7.2 Analysing a review

Enter a review in the text box and click Analyze (or press Enter). The application returns three outputs: the overall sentiment as a set of confidence bars over the negative, neutral, and positive classes; the detected language of the text; and a table listing each detected service aspect together with its sentiment and a confidence figure. For example, given the input:

Unifi fibre speed has been consistent for 6 months, but the bill is too expensive.

the application reports an overall mixed-to-negative sentiment, detects the language as English, and lists two aspects — speed (positive) and billing (negative) — reflecting the two opinions expressed in the

sentence. The interface also provides several ready-made example reviews, including Manglish examples, that can be loaded with a single click.

7.3 The deployed web application

A hosted copy of the application is published on Hugging Face Spaces, so it can be used without any local installation. Instructions for deploying or redeploying the application to Hugging Face Spaces — including how to upload the model and set the `HF_MODEL_ID` secret — are given in `deploy/DEPLOY.md`.

8 Input and Output Reference

This section documents the data formats the scripts expect and produce, which is useful when supplying new data or interpreting the results.

8.1 Data format

All modelling and evaluation code reads a comma-separated (CSV) file and uses two columns: `cleaned_text` holds the text to classify, and `sentiment_label` holds the ground-truth label, which is one of negative, neutral, or positive. Internally these labels are mapped to the integers 0, 1, and 2 respectively. The labelled datasets provided with the repository are:

File	Rows	Role
<code>data/labeled/labeled_main.csv</code>	1,200	Full labelled Play Store set
<code>data/labeled/labeled_main_train.csv</code>	960	Training split
<code>data/labeled/labeled_main_test.csv</code>	240	Held-out in-domain test split
<code>data/labeled/trustpilot_eval.csv</code>	60	Cross-platform evaluation tier
<code>data/labeled/forum_eval.csv</code>	60	Cross-domain evaluation tier

8.2 Output files

Running the pipeline produces the following artefacts under the `models/` directory:

File	Produced by	Contents
<code>models/baseline_lr.pkl</code> , <code>models/baseline_svm.pkl</code>	<code>baseline.py</code>	Trained baseline classifiers
<code>models/xlmr_final/</code>	<code>finetune_xlmr.py</code>	Fine-tuned transformer (config, weights, tokenizer)
<code>models/eval_results.csv</code>	<code>evaluate.py</code>	Three-tier accuracy comparison table
<code>models/aspect_degradation.csv</code>	<code>aspect_degradation.py</code>	Accuracy per model, tier, and aspect
<code>models/</code>	<code>aspect_degradation.py</code>	Heatmap images (if matplotlib installed)

aspect_degradation_*.png		
--------------------------	--	--

9 Troubleshooting

Symptom	Cause and remedy
A script cannot find a data or model file	The scripts use paths relative to the repository root. Make sure the current directory is the project root (the folder containing requirements.txt) when running any script.
evaluate.py reports a missing model	The evaluation needs a trained model. Run python src/baseline.py (and, for the transformer results, obtain models/xlmr_final/) before running the evaluation.
The web application says the model is not loaded	The fine-tuned model is not present at models/xlmr_final. Fine-tune the model (Section 6) or set the HF_MODEL_ID environment variable to a hosted model.
No heatmap images are produced	The heatmaps require matplotlib, whose import is optional. Install it with pip install matplotlib; the CSV output is written regardless.
The Trustpilot scraper fails	Trustpilot is protected by an automated-traffic challenge and requires the Chromium browser installed via python -m playwright install chromium. This affects data collection only, not the provided results.
The application is slow to start	On first use the transformer model (about 1.1 GB) is loaded into memory, which takes some time on CPU. Subsequent analyses are fast.

10 Glossary

Term	Meaning
Sentiment class	One of the three predicted categories: negative, neutral, or positive.
Aspect	One of the five fixed service topics the system recognises: coverage, speed, billing, customer support, and application usability.
In-domain / cross-platform / cross-domain	The three evaluation tiers: Play Store reviews (the training domain), Trustpilot reviews (a different platform), and forum posts (a different kind of text).
TF-IDF	Term Frequency–Inverse Document Frequency, the feature representation used by the baseline model.
XLM-RoBERTa (XLM-R)	A multilingual transformer language model, fine-tuned here as the advanced classifier.
Manglish	Informal code-switched Malay-English text common in Malaysian online reviews.
Degradation	The drop in accuracy as evaluation data moves further

	from the training domain; measuring it per aspect is the project's main finding.
--	--

11 Further Documentation

The repository contains additional documents for readers who need more detail:

Document	Contents
README.md	Project overview, team ownership, and the development workflow
METHODOLOGY.md	The technical approach and modelling decisions
EXPERIMENT_aspect_degradation.md	Full specification of the flagship degradation experiment
deploy/DEPLOY.md	Step-by-step deployment to Hugging Face Spaces
labeling/rubric.md	The labelling rubric with worked Malay and Manglish examples